

◆ 1. Externe Libraries installieren & verwalten

Am besten nutzt du **virtuelle Umgebungen**:

- Jede Projektmappe hat ihre eigene Umgebung mit genau den benötigten Paketen.
- So verhinderst du Chaos zwischen verschiedenen Projekten.

👉 Typischer Ablauf:

```
# Projektordner anlegen
mkdir meinprojekt && cd meinprojekt

# Virtuelle Umgebung erstellen
python3 -m venv .venv

# Aktivieren
source .venv/bin/activate    # Linux/Mac
.venv\Scripts\activate      # Windows

# Libraries installieren
pip install numpy requests flask

# Abhängigkeiten speichern
pip freeze > requirements.txt
```

Später kannst du mit

```
pip install -r requirements.txt
```

alles reproduzieren.

◆ 2. Eigene Libraries / Module organisieren

Wenn du selber Code als wiederverwendbare Library schreiben willst:

Projektstruktur

```
meinprojekt/
├── meinpaket/                # Dein Code als Package
│   ├── __init__.py          # macht es zu einem Package
│   ├── modul1.py
│   └── modul2.py
├── tests/                   # Tests
│   └── test_modul1.py
├── main.py                  # Einstiegspunkt
├── requirements.txt          # Abhängigkeiten
└── setup.py oder pyproject.toml # für Packaging
```

Beispiel meinpaket/modul1.py

```
def hallo(name: str):
    return f"Hallo {name}!"
```

Nutzung

```
from meinpaket.modul1 import hallo  
  
print(hallo("Bernhard"))
```

◆ 3. Wann lohnt sich Packaging?

- Wenn du **denselben Code in mehreren Projekten** nutzen willst.
- Dann erstellst du ein **PyPI-Paket** oder installierst lokal:

```
pip install -e .
```

im Projektordner mit `setup.py` oder `pyproject.toml`.

👉 Meine Empfehlung für dich als Embedded-SW-Entwickler:

- Für **jedes Projekt eine venv**
- **Requirements-Datei** führen
- Eigene Libs in **eigenen Packages mit klarer Struktur**